GUIDE DE COMPRÉHENSION DU CODE

Gestion des étudiants avec React

Explication détaillée de l'ajout, de la modification, de la suppression et du filtrage des étudiants, ainsi que des principales syntaxes React utilisées dans le projet.

Ce guide explique

- le rôle de `useState`, des props et du JSX ;
- comment ajouter, modifier et supprimer une fiche ;
- comment filtrer les fiches selon la filière ;
- comment les composants `App` et `StudentCard` communiquent ;
- les tests à faire et les limites actuelles du stockage.

Document rédigé en français à partir du code actuel du projet.

1. Vue d'ensemble de l'application

La page est organisée autour de deux composants principaux. `App` conserve la liste des étudiants et réalise les opérations sur cette liste. `StudentCard` affiche un étudiant et ses boutons d'action.

Fichier / élément	Responsabilité
`App.jsx`	Gère l'état global des étudiants, l'ajout, la suppression, la modification et le filtre.
`StudentCard.jsx`	Affiche une fiche individuelle, ses coordonnées et le formulaire de modification.
`App.css`	Définit la présentation des cartes, du filtre et du formulaire de modification.

Les imports React et les composants

```
import { useState } from 'react';
import StudentCard from './components/StudentCard.jsx';
```

`import` rend disponibles dans ce fichier des éléments venant d'autres fichiers ou de React. `useState` permet de conserver des données qui changent. `StudentCard` est le composant enfant qui représente une fiche.

Le JSX

Le JSX est une syntaxe utilisée avec React pour décrire les éléments à afficher en écrivant une forme proche du HTML dans JavaScript. Les expressions JavaScript sont placées entre accolades : par exemple `{'{etudiant.nompr}'}`. En JSX, on écrit `className` au lieu de `class`, et les éléments doivent être correctement fermés.

La liste initiale

```
const [etudiants, setEtudiants] = useState([
  {
    nompr: 'Ranya Chebli',
    email: 'ranya@example.com',
    tel: '20815709',
    filiere: 'Génie logiciel',
    groupe: 'Glsi 2 B',
    ville: 'Nabeul',
    image: MyImage,
  },
]);
```

`etudiants` est la valeur courante de l'état : ici, le tableau d'étudiants. `setEtudiants` est la fonction à appeler pour demander à React de mettre à jour cet état. Chaque étudiant est un objet JavaScript contenant ses propriétés. Les données de l'exemple dans le projet sont fournies au démarrage.

2. Les notions React et JavaScript utilisées

État et rendu

Un **état** est une donnée mémorisée par un composant. Lorsqu'un setter tel que `setEtudiants` ou `setIsEditing` est appelé avec une nouvelle valeur, React réaffiche les éléments qui dépendent de cette valeur.

```
const [valeur, setValeur] = useState(valeurInitiale);
```

- `valeur` : valeur actuelle ;
- `setValeur` : fonction de mise à jour ;
- `valeurInitiale` : valeur de départ.

Props : transmettre des données et des fonctions

Les **props** sont les informations transmises d'un composant parent à un composant enfant. Dans `App.jsx`, chaque fiche reçoit par exemple `nompr`, `email`, `image`, ainsi que `onDelete` et `onSave`. Dans `StudentCard`, ces valeurs sont lues avec `props.nompr`, `props.email`, etc.

```
<StudentCard
  nompr={etudiant.nompr}
  onDelete={() => supprimerEtudiant(etudiant)}
/>
```

Les accolades indiquent que les valeurs entre accolades sont des expressions JavaScript. La fonction fléchée permet d'attendre le clic avant d'appeler la fonction de suppression.

Fonctions fléchées et callbacks de mise à jour

```
(precedents) => [...precedents, nouvelEtudiant]
```

`=>` définit une fonction fléchée. Quand une mise à jour dépend de l'état précédent, React permet de lui passer une fonction : cette fonction reçoit la valeur précédente. Cette forme est utilisée pour ajouter, modifier ou filtrer une liste de façon fiable.

Décomposition et copie d'objets/tableaux

- `[...precedents, nouvelEtudiant]` crée un nouveau tableau avec les anciens éléments et le nouvel étudiant à la fin.
- `{'{ ...e, ...informationsModifiees }'}` crée un nouvel objet : les anciennes propriétés sont copiées, puis les propriétés modifiées les remplacent.
- `{'{ ...precedentes, [name]: value }'}` conserve les autres champs et met à jour celui dont le nom est contenu dans `name`.

> En React, on évite de modifier directement le tableau ou l'objet existant. On en crée une copie, puis on transmet cette nouvelle valeur au setter.

Comparaison stricte et conditions

`===` compare la valeur et le type sans conversion automatique. `!==` signifie « différent de manière stricte ». L'opérateur `!` inverse une valeur booléenne. L'opérateur `||` signifie « ou » ; `&&` signifie « et ».

L'opérateur ternaire `condition ? valeurA : valeurB` choisit entre deux résultats. React utilise aussi les expressions `condition && (<Element />)` pour afficher un élément seulement si la condition est vraie.

3. Ajouter un étudiant

État qui mémorise le contenu du formulaire

```
const [nouvelEtudiant, setNouvelEtudiant] = useState({
  nompr: '',
  email: '',
  tel: '',
  filiere: '',
  groupe: '',
  ville: '',
});
```

Chaque propriété correspond à un champ du formulaire. Les champs sont initialisés à une chaîne vide. L'image est attribuée ensuite par le code d'ajout.

Réagir à la saisie

```
function handleChange(event) {
  const { name, value } = event.target;
  setNouvelEtudiant((precedent) => ({
    ...precedent,
    [name]: value,
  }));
}
```

1. Le navigateur appelle `handleChange` quand la valeur d'un champ change.
2. `event.target` désigne le champ concerné.
3. `name` est la valeur de l'attribut `name` du champ ; `value` est le texte saisi.
4. La décomposition copie les autres données, puis `[name]: value` met à jour uniquement le champ concerné. Les crochets indiquent une propriété calculée.

Les champs du JSX sont contrôlés par React : leur `value` vient de `nouvelEtudiant`, et `onChange` appelle `handleChange`.

Enregistrer le nouvel étudiant

```
function ajouterEtudiant(event) {
  event.preventDefault();
  if (!nouvelEtudiant.nompr ||
      !nouvelEtudiant.email ||
      !nouvelEtudiant.tel) {
    return;
  }

  const etudiantAjoute = {
    ...nouvelEtudiant,
    image: MyImage,
  };
  setEtudiants((precedents) => [...precedents, etudiantAjoute]);
  setNouvelEtudiant({
    nompr: '', email: '', tel: '',
    filiere: '', groupe: '', ville: '',
```

```
  });
}
```


- `preventDefault()` empêche le comportement normal du formulaire, qui rechargerait la page.
- Le `if` vérifie que nom, email et téléphone ne sont pas vides ; `return` arrête l'ajout si une valeur manque.
- La décomposition copie tous les champs et ajoute une image par défaut.
- Le nouveau tableau est créé en ajoutant l'objet à la fin de la liste précédente.
- Le formulaire est ensuite réinitialisé avec des chaînes vides.

Le formulaire utilise `onSubmit={ajouterEtudiant}` et le bouton utilise `type="submit"` : l'envoi appelle donc la fonction d'ajout.

4. Modifier les informations

État local de la fiche

Dans `StudentCard`, `isEditing` indique si le formulaire est ouvert. `informations` conserve les valeurs temporaires du formulaire. `showContact` indique si les coordonnées détaillées sont visibles.

```
const [isEditing, setIsEditing] = useState(false);
const [informations, setInformations] = useState({
  nompr: '', email: '', tel: '',
  filiere: '', groupe: '', ville: '',
});
```

Initialiser les champs avec la fiche courante

```
function commencerModification() {
  setInformations({
    nompr: props.nompr,
    email: props.email,
    tel: props.tel,
    filiere: props.filiere,
    groupe: props.groupe,
    ville: props.ville,
  });
  setIsEditing(true);
}
```

Cette fonction copie les données actuelles reçues dans les props vers l'état du formulaire, puis ouvre celui-ci. L'utilisateur voit ainsi les valeurs existantes et peut changer une ou plusieurs d'entre elles.

Les champs décrits par une liste

Le tableau `champs` associe à chaque champ son nom, son libellé et son type. `champs.map(...)` parcourt ce tableau pour générer les balises `label` et `input`. L'attribut `key` donne à React une identité stable pour chaque élément créé par une liste.

```
<input
  type={champ.type}
  name={champ.name}
  value={informations[champ.name]}
  onChange={handleChange}
  required={['nompr', 'email', 'tel'].includes(champ.name)}
/>
```

- `value` relie le champ à son état : React contrôle sa valeur.
- `onChange` met l'état à jour pendant la saisie.
- `required` rend le nom, l'email et le téléphone obligatoires dans le formulaire de modification.
- `includes` vérifie si le nom du champ fait partie du tableau donné.

Enregistrer et annuler

```
function enregistrerModification(event) {
  event.preventDefault();
  props.onSave(informations);
  setIsEditing(false);
}
```


La fonction empêche le rechargement, transmet les valeurs au parent via `onSave` et ferme le formulaire. Le bouton « Annuler » ferme le formulaire sans appeler `onSave`, donc les changements saisis ne sont pas appliqués à la liste.

5. Supprimer un étudiant


```
function supprimerEtudiant(etudiantASupprimer) {
  setEtudiants((precedents) =>
    precedents.filter(
      (etudiant) => etudiant !== etudiantASupprimer
    )
  );
}
```

`filter` construit un nouveau tableau en gardant uniquement les éléments pour lesquels la condition est vraie. Ici, les étudiants qui ne sont pas l'objet sélectionné sont conservés ; l'objet à supprimer est exclu.

Le bouton de la fiche appelle `props.onDelete`. Dans `App.jsx`, cette prop est reliée à `supprimerEtudiant(etudiant)`. La carte ne modifie donc pas elle-même la liste : elle demande au parent de le faire.

```
<button type="button" onClick={props.onDelete}>
  Supprimer étudiant
</button>
```

> Dans le code actuel, la suppression est immédiate : il n'y a pas de boîte de confirmation. Une fois la page actualisée, la liste initiale revient, car les données ne sont pas enregistrées dans une base de données.

6. Filtrer par filière

Construire les options du menu

```
const [filiereChoisie, setFiliereChoisie] = useState('');
const filieres = [
  ...new Set(etudiants.map((etudiant) => etudiant.filiere))
];
```

`filiereChoisie` garde la valeur sélectionnée dans le menu. La chaîne vide signifie qu'aucun filtre n'est actif. `map` extrait la filière de chaque étudiant. `Set` retire les doublons, et `...` transforme le résultat en tableau. Ainsi, le menu est généré à partir des filières présentes dans la liste, y compris après un ajout ou une modification.

Calculer et afficher les résultats

```
const etudiantsFiltres = etudiants.filter(
  (etudiant) =>
    filiereChoisie === '' ||
    etudiant.filiere === filiereChoisie
);
```

Si aucune filière n'est choisie, la première condition est vraie et tous les étudiants sont conservés. Sinon, seuls ceux dont la filière correspond exactement au choix restent dans la liste.


```
<select
  value={filiereChoisie}
  onChange={(event) => setFiliereChoisie(event.target.value)}
>
  <option value="">Toutes les filières</option>
  {filieres.map((filiere) => (
    <option key={filiere} value={filiere}>
      {filiere}
    </option>
  ))}
</select>

{etudiantsFiltres.map((etudiant) => (
  <StudentCard key={...} ... />
))}
```

Le menu est contrôlé par React : `value` reflète l'état et `onChange` le met à jour. L'affichage parcourt `etudiantsFiltres`, pas la liste complète.

7. Communiquer entre App et StudentCard

Le parent possède les données et transmet chaque étudiant à une carte. Il transmet aussi des fonctions de rappel afin que la carte puisse demander une action au parent.

```
{`{etudiantsFiltres.map((etudiant) => (
   supprimerEtudiant(etudiant)}
    onSave={(informationsModifiees) =>
      modifierEtudiant(etudiant.email, informationsModifiees)
    }
  />
))}`}
```

- `map` crée une carte pour chaque étudiant affiché.
- `key` aide React à reconnaître chaque élément de la liste quand celle-ci change.
- Les props simples transmettent les données à montrer.
- `onDelete` transmet la demande de suppression.
- `onSave` transmet les données modifiées au parent.

Modifier la liste dans le parent

```
function modifierEtudiant(emailInitial, informationsModifiees) {
  setEtudiants((precedents) =>
    precedents.map((etudiant) =>
      etudiant.email === emailInitial
        ? { ...etudiant, ...informationsModifiees }
        : etudiant
    )
  );
}
```

`map` examine chaque étudiant. Si son email correspond à l'identifiant mémorisé avant la modification, le code crée un objet avec les anciennes propriétés et les nouvelles valeurs. Sinon, il conserve l'objet tel quel. `===` est une comparaison stricte (valeur et type). Le parent réaffiche ensuite les cartes avec les informations actualisées.

Afficher des éléments selon une condition

`{'{!isEditing && (...)}'}` affiche le mode normal uniquement quand on ne modifie pas. `{'{isEditing && (...)}'}` affiche le formulaire uniquement pendant la modification. Le `!` signifie « non », et `&&` signifie que le contenu de droite s'affiche si la condition de gauche est vraie.

8. Les styles CSS et les éléments déjà présents

Sélecteur CSS	Rôle
`.fiche`	Présente les fiches dans une grille avec espacement et largeur maximale.
`.fiche > *`	Applique bordure, coins arrondis et marges intérieures aux éléments directs de la grille.
`.fiche button`	Donne un style commun aux boutons des fiches.
`.supprimer-button`	Applique une couleur rouge au bouton de suppression.
`.modification-form`	Organise le formulaire de modification en grille.
`.modification-form input`	Étend les champs à la largeur disponible et ajoute de l'espace intérieur.
`.filtre-filiere`	Centre le libellé et le menu de filtre.

Dans CSS, un point devant le nom sélectionne une classe (`.fiche`). `display: grid` crée une grille ; `display: flex` aligne des éléments dans une rangée ou une colonne selon ses règles.

Composants déjà présents, mais indépendants des nouveautés

`Header`, `Navbar`, `Footer` et `Counter` sont également affichés par `App`. Le compteur possède son propre état et son propre bouton d'incrémentation. Ces éléments ne sont pas les mécanismes d'ajout, de modification, de suppression ou de filtrage des étudiants.

L'année « 2026/2027 » affichée dans le détail de la carte est du texte fixe dans le composant. Elle ne se met pas à jour automatiquement.

9. Parcours pratique et vérifications

Ajouter

1. Remplir le nom, l'email et le téléphone (les champs filière, groupe et ville peuvent aussi être renseignés).
2. Cliquer sur « Ajouter Etudiant ».
3. Vérifier que la nouvelle fiche apparaît et que le formulaire est vidé.
4. La filière du nouvel étudiant devient automatiquement une option du filtre.

Modifier

1. Cliquer sur « Modifier les informations » sur la fiche voulue.
2. Changer une ou plusieurs valeurs dans le formulaire déjà rempli.
3. Cliquer sur « Enregistrer » pour appliquer les changements, ou « Annuler » pour les abandonner.
4. Si la filière est modifiée, les options du filtre se recalculent à partir de la liste mise à jour.

Filtrer

1. Choisir une filière dans le menu pour n'afficher que les étudiants concernés.
2. Choisir « Toutes les filières » pour retirer le filtre.

Supprimer

1. Cliquer sur « Supprimer étudiant » sur la fiche choisie.
2. La fiche disparaît immédiatement et la liste du filtre est recalculée.

10. Limites importantes

> Les étudiants sont conservés uniquement dans l'état React en mémoire. Les ajouts, modifications et suppressions ne sont pas enregistrés dans une base de données ni dans le navigateur. Un rechargement de la page réinitialise la liste aux étudiants codés au départ.

- Il n'y a pas de confirmation avant la suppression.
- L'image d'un nouvel étudiant est l'image par défaut `MyImage` ; le formulaire ne permet pas de choisir une image.
- La validation de l'ajout vérifie que nom, email et téléphone ne sont pas vides. Le champ email HTML utilise aussi `type="email"`.
- La modification utilise l'email initial comme identifiant pour retrouver l'étudiant, même si l'email lui-même est modifié pendant l'opération.

Résumé

App détient et met à jour le tableau des étudiants. **StudentCard** présente une fiche et déclenche des actions via des props. **useState** conserve les données réactives, les méthodes **map** et **filter** construisent les vues adaptées, et le **JSX** décrit ce que React affiche.